

Parsek Flight Recorder — Design Document

Comprehensive design specification for Parsek's vessel recording, playback, multi-vessel tracking, and timeline system — including the vessel interaction paradox resolution, ghost chain model, relative-state time jump, and spawn safety.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones.

1. Introduction

This document specifies the full architecture of Parsek's flight recorder system. It covers:

- The recording tree model (DAG structure) and vessel identity
- Segment boundary rule (only structural separation creates segments)
- Within-segment events for controller changes, part destruction, etc.
- Crash breakup coalescing
- Multi-vessel recording sessions with focus switching
- Segment taxonomy (environment, reference frame, rendering zones)
- Background vessel recording in the physics bubble
- Orbital checkpoint system for on-rails vessels
- Looped recording playback anchored to real vessels
- Multi-vessel merge to a single committed timeline
- Interaction with pre-existing persistent vessels and the vessel interaction paradox
- Ghost chain resolution: ghosting, spawn suppression, chain-tip spawning
- Spawn safety: collision detection, ghost extension, terrain correction, trajectory walkback
- Relative-state time jump for preserving rendezvous geometry across ghost windows
- Selective spawning UI for choosing which ghosts to materialize
- Ghost world presence: tracking station, map view, CommNet relay
- Distance-based rendering zones and performance budget
- Rewind and fast-forward with ghost playback
- Ghost implementation constraints
- Recording file format
- Error recovery and backward compatibility

Terminal-state and synthetic-tail reliability across scene exit, vessel unload/delete, crash, and background recording end paths is specified in [parsek-recording-finalization-design.md](#). That companion document supersedes older scene-exit-only assumptions whenever a recording can end before a live vessel is available for finalization.

2. Design Philosophy

These principles govern every design decision in the flight recorder. They are listed here because they inform every section that follows.

2.1 Recording and Playback

1. **Controller-based identity.** Vessel identity follows command authority, not KSP's unstable persistentId. A vessel is defined by its controllers, not its root part.
2. **Structural separation only.** Segment boundaries occur only when the vessel physically splits or merges. Controller changes, part destruction without separation, and crew movement within a vessel are

SegmentEvents within a continuing segment. A player who launches a vessel, partially crashes it, and recovers the wreckage sees ONE recording — not multiple disconnected segments.

3. **Highest fidelity wins.** Multiple overlapping data sources are merged by selecting the best at each UT: active > background > checkpoint.
4. **Additive format evolution.** Recording formats add fields, never remove or rename them. Old recordings play at reduced fidelity. New recordings carry extra data that old versions ignore.

2.2 Ghost Chains and Paradox Prevention

5. **Ghosts are the only reliable paradox prevention.** UI blocks and reservation systems can be bypassed by physics. A ghost vessel has no physics, no colliders, no docking ports — physical interaction is impossible by construction.
6. **Ghost until tip of chain.** A vessel claimed by a committed recording stays ghost until the final committed interaction in its lineage completes. The real vessel spawns only at the chain tip.
7. **Physical state is the trigger.** Only events that change a vessel's orbit, part configuration, resources, or crew force ghosting. Observation and cosmetic changes do not.
8. **Time warp is the workaround.** Players time-warp past ghost windows. In stock KSP with no time-dependent consumables, this has zero gameplay cost. The relative-state time jump (Section 14.5) eliminates the rendezvous-drift cost.
9. **First loop is real, rest are visual.** Looped recordings only spawn real vessels on the first iteration. Subsequent loops are ghost-only scenery. This prevents a looped docking recording from permanently ghosting a station.
10. **No deletion, no paradox.** Committed recordings are permanent. The ghost chain can only grow (new recordings added), never shrink (recordings removed). This guarantees monotonic convergence to a consistent timeline.
11. **Physical interactions and resource effects are decoupled.** Ghost chains control what the player can touch. The game actions timeline controls what resources exist. Neither system depends on the other. Ghosting a vessel does not affect its resource history.
12. **Paradox prevention is enforced by the physics engine, not by validation logic.** The player cannot create a conflicting recording because conflicting interactions are physically impossible — ghost vessels have no colliders, no docking ports, no physics presence. This is not a rule that can be bypassed by mods, exploits, or the debug menu. It is a property of the simulation itself.

2.3 Time Jump

The time jump is a specialized mechanism for **multi-vessel rendezvous configurations** where the player has set up an approach to a ghosted vessel and needs to skip past its chain tip without losing the alignment. Normal time warp (Section 14.4) remains the default way to advance time — the timeline plays out, ghosts appear and despawn, vessels spawn at their endpoints. The time jump exists only because normal warp destroys carefully aligned approach geometry through differential Keplerian drift.

13. **Nothing moves, the clock jumps.** During a time jump, the entire physics bubble stays frozen in place. Only UT changes. Orbital epochs are adjusted to maintain Keplerian consistency. This eliminates drift by construction.

14. **Selective spawning, chronological constraint.** The player picks which chain tip to jump to. Earlier independent tips are auto-spawned (ghosts cannot exist past their tip). Later tips stay ghost.
15. **Time jump complements warp, does not replace it.** Fast-forward via normal time warp is always available and plays out the timeline naturally. The time jump is an additional tool specifically for preserving bubble geometry when the player needs to interact with a vessel that is currently ghost.
16. **Visual honesty.** Planet rotation, sun angle, and star field change to reflect the new UT. The player sees that time passed. Only the local bubble geometry is preserved.
17. **No persistent side effects.** The time jump is equivalent to a warp — it advances UT and processes the timeline. Rewind undoes it normally. Recordings are not modified.
18. **Game actions processed atomically.** All resource recalculation happens once at the target UT, same as warp exit.

2.4 Ghost World Presence

19. **Ghosts participate selectively.** Ghosts are lightweight (raw GameObjects), not full KSP Vessels. They participate in CommNet and map view through selective API registration. They do not participate in any active game system. This preserves the paradox-proof boundary while allowing the game world to feel correct.
-

3. Vessel Identity Model

3.1 Why KSP's Vessel ID Is Insufficient

KSP assigns each vessel a `persistentId`. This ID is unstable across vessel lifecycle events: when a vessel decouples, one side inherits the original ID (the side with the root part) and the other gets a new ID. When two vessels dock, one ID survives and the other is destroyed. The surviving ID depends on which vessel was active at docking time.

This means a vessel that launches as one craft, drops stages, undocks a lander, docks to a station, and later gets recycled will have its KSP vessel ID change multiple times in ways that don't correspond to what the player considers "the same thing."

3.2 Controller-Based Identity

Parsek defines vessel identity around **command authority** — the presence of a controller part that gives a vessel agency.

Controller parts (in priority order):

- Crewed command pod (Mk1, Mk1-3, etc.)
- Occupied external command seat
- Probe core (with or without electric charge)
- Kerbal on EVA

A vessel segment is the continuous history of a command-capable assembly of parts, from the moment it gains command authority to the moment it loses it permanently.

Gains command authority: A controller part becomes the root of an independent vessel — either at launch, at a split event where this side has a controller, or via construction (a kerbal places and activates a probe core).

Loses command authority permanently: Recovery, destruction, recycling, or removal of all controller parts with no replacement.

3.3 Debris

An assembly of parts with no controller is debris. Parsek does not record debris trajectories. KSP's own persistence handles debris positions.

If a controlled vessel acquires debris (e.g., claw grab of a fuel tank), that's a merge event where one parent is a controller-based vessel and the other is inert. The recording notes the acquisition; the debris's prior existence is irrelevant to the recording.

If a vessel loses all controllers (crew removed, probe core out of EC with no restoration), its active segment ends and it becomes debris. If a controller is later restored (another vessel docks and provides control, a kerbal boards it, EC is restored to a probe core), a new active segment begins.

4. The Recording Tree (DAG)

4.1 Structure

A recording tree is a **directed acyclic graph (DAG)** of vessel segments connected by lifecycle events. It is not a simple tree because merge events (docking, boarding) join branches back together.

4.2 Terminology

This document uses two layers of naming. The **design model** uses conceptual names (VesselSegment, TreeEvent) to describe what the DAG represents. The **implementation** uses concrete class names (Recording, BranchPoint) that appear in code. The mapping:

Design concept	Implementation class	Meaning
VesselSegment	Recording	One vessel's continuous trajectory + events
TreeEvent	BranchPoint	A DAG node where vessels split, merge, launch, or terminate
RecordingTree	RecordingTree	The complete DAG for one mission

Throughout this document, "Recording" (capitalized) refers to the per-vessel data object. "Committed tree" or "mission recording" refers to a RecordingTree that has been committed to the timeline. The word "recording" (lowercase) refers to the act of recording.

4.3 Authoritative Data Model

These are the actual data structures. All types referenced elsewhere in this document are defined here.

RecordingTree — the complete DAG for one mission. Committed trees are stored in a flat list (`RecordingStore.CommittedTrees`) that survives scene changes via static field persistence and is serialized/deserialized by the ScenarioModule (`ParsekScenario`). The ghost chain walker iterates this list on every rewind and save load.

```
RecordingTree
  Id:                string – unique identifier
  TreeName:          string
  RootRecordingId:   string – the first Recording (launch)
  ActiveRecordingId: string – currently focused Recording (changes on focus switch)
  Recordings:        dict of {recordingId: Recording} – all vessel recordings in this tree
  BranchPoints:      list of BranchPoint – DAG events connecting Recordings
```

BackgroundMap:	dict of {vesselPid: recordingId} – background vessel mappings (runtime only)
PreTreeFunds:	double – game state at tree start (for delta computation)
PreTreeScience:	double
PreTreeReputation:	float
DeltaFunds:	double – net resource change from this tree
DeltaScience:	double
DeltaReputation:	float
ResourcesApplied:	bool

Recording — one vessel's continuous trajectory, events, and state. This is the VesselSegment of the design model. A Recording covers the lifetime of one command-capable assembly from creation to termination (or ongoing).

```
Recording
  RecordingId:          string – unique identifier
  TreeId:              string – which RecordingTree this belongs to
  VesselName:          string
  VesselPersistentId:  uint – KSP's vessel PID at recording time
  RecordingFormatVersion: int – sidecar file version (currently 6, 7 with ghost chain
fields)

  // Trajectory data (new format)
  TrackSections:        list of TrackSection – typed trajectory chunks (Section 6)
  // Trajectory data (legacy format – dual-written for backward compat with pre-TrackSection
v5 recordings)
  Points:              list of TrajectoryPoint – flat trajectory
  OrbitSegments:       list of OrbitSegment – flat orbit segments

  // Events
  PartEvents:          list of PartEvent – discrete visual state changes (35 event types)
  SegmentEvents:       list of SegmentEvent – within-segment state changes (Section 4.8)
  Controllers:         list of ControllerInfo – controller parts at segment start
  IsDebris:            bool – true if no controllers

  // DAG linkage
  ParentBranchPointId: string – BranchPoint that created this Recording
  ChildBranchPointId:  string – BranchPoint that ended this Recording
  ParentRecordingId:   string – parent in the tree

  // Terminal state (how/where the vessel ended)
  TerminalStateValue:  RECOVERED | DESTROYED | RECYCLED | DESPAWNED | null
  TerminalOrbit*:      Keplerian elements at segment end (inc, ecc, sma, lan, argPe, mAe,
epoch, body)
  TerminalPosition:    SurfacePosition – lat/lon/alt if surface-landed at end
  TerrainHeightAtEnd:  double – terrain height at final position (NaN if not surface; for
spawn correction)

  // Vessel snapshots (ConfigNode from KSP's vessel.BackupVessel() – complete vessel state)
  VesselSnapshot:      ConfigNode – full vessel state: part tree, resources, crew, orbit,
action groups
  GhostVisualSnapshot: ConfigNode – visual-only snapshot for ghost mesh building
```

```

// Loop configuration
LoopPlayback:          bool
LoopIntervalSeconds:   double
LoopAnchorVesselId:    uint – anchor vessel for relative loop positioning
LoopAnchorBodyName:    string – expected body (for drift validation)

// Spawn tracking
VesselSpawned:         bool – has this Recording's vessel been spawned into the game?
SpawnedVesselPersistentId: uint – PID of the spawned vessel (0 = not spawned)

// Antenna data (for CommNet ghost registration)
AntennaSpecs:          list of AntennaSpec

```

VesselSnapshot explained: A snapshot is a KSP `ConfigNode` produced by `vessel.BackupVessel()`. It contains the complete serialized vessel state: the entire part tree with every module's persistent data, crew assignments, resource levels, orbital elements, vessel situation, action group states, and discovery info. Snapshots are captured at two points: (1) at recording commit time (stored on the Recording), and (2) at ghost conversion time (captured from the live vessel before despawn). For chain-tip spawning, the snapshot from the tip Recording is used — for a chain bare-S -> S+A -> S+A+B, the S+A+B snapshot comes from R2's Recording of the merged vessel at its endpoint.

Finalization note: `TerminalStateValue`, `ExplicitEndUT`, terminal orbit/surface metadata, and predicted `OrbitSegment` tails must be finalized consistently even if KSP destroys or unloads the vessel before scene exit. The reliability contract and cache design are documented in [parsek-recording-finalization-design.md](https://github.com/ParsecProject/Parsec/blob/master/docs/parsek-recording-finalization-design.md).

BranchPoint — a DAG node where the tree branches or merges. This is the `TreeEvent` of the design model.

```

BranchPoint
  Id:                string – unique identifier
  UT:                double – universal time
  Type:              BranchPointType enum:
                     Undock=0, EVA=1, Dock=2, Board=3, JointBreak=4,
                     Launch=5, Breakup=6, Terminal=7
  ParentRecordingIds: list of string – Recordings that ended here
  ChildRecordingIds:  list of string – Recordings that began here

// Type-specific metadata (nullable, set based on Type)
SplitCause:          DECOUPLE | UNDOCK | EVA
DecouplerPartId:     uint – part that triggered separation
BreakupCause:         CRASH | OVERHEAT | STRUCTURAL_FAILURE
BreakupDuration:     double – time window of the breakup
DebrisCount:         int – non-tracked fragment count
CoalesceWindow:      double – threshold used (default 0.5s)
MergeCause:          DOCK | BOARD | CONSTRUCT | CLAW
TargetVesselPersistentId: uint – pre-existing vessel's PID (links to game state; used by
chain walker)
TerminalCause:       RECOVERED | DESTROYED | RECYCLED | DESPAWNED

```

TrajectoryPoint — position sample. Recorded at up to 50 Hz in atmospheric flight, down to 0.1 Hz for surface-stationary. The most-sampled data structure in the system.

```

TrajectoryPoint
  ut:                double – universal time

```

```

latitude: double – body-relative
longitude: double – body-relative
altitude: double – above sea level
rotation: Quaternion – vessel attitude
velocity: Vector3 – vessel velocity
bodyName: string – reference body
funds: double – game state at this moment (for resource delta tracking)
science: float
reputation: float

```

OrbitSegment — Keplerian orbital elements for on-rails coasting. Used within ORBITAL_CHECKPOINT TrackSections and as the legacy flat orbit list.

```

OrbitSegment
  startUT: double
  endUT: double
  inclination: double
  eccentricity: double
  semiMajorAxis: double
  longitudeOfAscendingNode: double
  argumentOfPeriapsis: double
  meanAnomalyAtEpoch: double
  epoch: double
  bodyName: string
  orbitalFrameRotation: Quaternion
  angularVelocity: Vector3

```

TrackSection — a typed trajectory chunk with a single environment + reference frame combination.

```

TrackSection
  environment: SegmentEnvironment (ATMOSPHERIC | EXO_PROPULSIVE | EXO_BALLISTIC
  |
  SURFACE_MOBILE | SURFACE_STATIONARY)
  referenceFrame: ReferenceFrame (ABSOLUTE | RELATIVE | ORBITAL_CHECKPOINT)
  source: TrackSectionSource (ACTIVE | BACKGROUND | CHECKPOINT)
  startUT: double
  endUT: double
  anchorVesselId: uint – set only for RELATIVE frame
  frames: list of TrajectoryPoint – for ABSOLUTE and RELATIVE
  checkpoints: list of OrbitSegment – for ORBITAL_CHECKPOINT
  sampleRateHz: float – actual recording sample rate
  boundaryDiscontinuityMeters: float – position gap vs previous section end

```

PartEvent — a discrete visual state change on a specific part. 35 event types covering engines, parachutes, solar panels, antennas, lights, landing gear, cargo bays, fairings, RCS, robotics, thermal animations, inventory deployables.

```

PartEvent
  ut: double
  partPersistentId: uint – which part
  eventType: PartEventType (EngineIgnited, EngineShutdown, EngineThrottle,
  DeployableExtended, DeployableRetracted, LightOn, LightOff,
  GearDeployed, GearRetracted, ParachuteDeployed, ParachuteCut,

```

```

CargoBayOpened, CargoBayClosed, FairingJettisoned, RCSActivated,
RCSStopped, Decoupled, Destroyed, Docked, Undocked, ... 35 total)
partName:      string
value:         float – event-specific (e.g., throttle percentage)
moduleIndex:   int – which module on the part (for multi-module parts)

```

SegmentEvent — see Section 4.8.

ControllerInfo — descriptor for a controller part.

```

ControllerInfo
type:          ControllerType (CrewedPod | ExternalSeat | ProbeCore | EVAKerbal)
partName:      string
partPersistentId: uint

```

AntennaSpec — antenna data for CommNet ghost registration.

```

AntennaSpec
partName:      string
antennaPower:  double – from ModuleDataTransmitter
antennaCombinable: bool
antennaCombinableExponent: double

```

4.4 The Segment Boundary Rule

CRITICAL: Only physical structural separation creates new segments. Controller changes never create segment boundaries.

A segment boundary (TreeEvent) occurs **ONLY** when:

- The vessel physically splits into two or more independent assemblies (SPLIT, BREAKUP)
- The vessel physically merges with another assembly (MERGE)
- The vessel ceases to exist (TERMINAL)
- The vessel is created (LAUNCH)

A segment boundary does **NOT** occur when:

- A controller part is destroyed but the vessel remains one connected assembly
- A controller part loses power, hibernates, or becomes non-functional
- A controller part regains power or is repaired
- The player uses "Control From Here" to change the active control point
- KSP internally reassigns the root part
- Crew transfers between parts within the same vessel
- Parts are destroyed, added, or removed without splitting the vessel

These non-structural changes are recorded as SegmentEvents (Section 4.8) within the continuing segment.

4.5 Identity Persistence Rule

A vessel segment persists as long as at least one controller part **physically exists** on the vessel, regardless of whether that controller is currently **functional**.

- A probe core with no electric charge is still a controller (it physically exists).
- A command pod with no crew is still a controller (the part exists).
- A hibernating probe core is still a controller.

- A damaged but not destroyed controller is still a controller.

Only physical destruction or removal of ALL controller parts ends the segment.

4.6 Split Events

When a vessel physically separates into independent assemblies (staging, undocking, EVA), the parent segment ends and two or more child segments begin. Each child inherits whatever controller parts end up on its side. Children without any controller are tagged as debris and receive minimal recording (position only until destruction/despawn).

4.7 Breakup Events (Crash Coalescing)

Crashes, overheating, and structural failures can produce many fragments across multiple physics frames. Rather than recording 20 individual SPLIT events in rapid succession, Parsek coalesces all separation events within a short time window (default 0.5 seconds) into a single BREAKUP tree event.

The BREAKUP event records the parent segment, all child segments that have controllers (these get proper segments with trajectory recording), and a count of debris fragments (not individually tracked).

Example — Mun crash with partial survival:

Vessel with probe core A (front) and probe core B (rear) crashes.
Multiple parts break off across 0.3 seconds.

Parsek records:

```
BREAKUP at T+0.00 (duration: 0.30s)
  parent: [original vessel]
  controlledChildren:
    - [assembly containing probe core A] -> new segment
    - [assembly containing probe core B] -> new segment
  debrisCount: 14
```

4.8 Within-Segment Events (SegmentEvents)

SegmentEvents record significant state changes that do NOT create DAG branches. They are annotations on a continuing segment, important for crew tracking, ghost visual fidelity, and vessel state history.

```
SegmentEvent
  type:      CONTROLLER_CHANGE | CONTROLLER_DISABLED | CONTROLLER_ENABLED |
             CREW_LOST | CREW_TRANSFER | PART_DESTROYED | PART_REMOVED | PART_ADDED |
             TIME_JUMP
  ut:        universal time
  details:   type-specific data
```

Types:

- **CONTROLLER_CHANGE** — A controller part is destroyed, added, or changes type. Lists lost controllers, gained controllers, and remaining controllers. The segment continues.
- **CONTROLLER_DISABLED** — Controller exists but is non-functional (no EC, hibernation, damaged). Segment continues. Identity persists.
- **CONTROLLER_ENABLED** — Controller regains functionality (EC restored, repaired, wake from hibernation).
- **CREW_LOST** — Crew member died (destroyed or MIA). Relevant for crew tracking.
- **CREW_TRANSFER** — Crew moved between parts within the same vessel (not via EVA — EVA is a SPLIT/MERGE in the DAG). Records kerbal name, from-part, to-part.

- **PART_DESTROYED** — Part destroyed without splitting the vessel. Ghost stops rendering this part mesh after this UT. Records part name, part ID, whether it was a controller.
- **PART_REMOVED** — Part removed via construction without splitting the vessel.
- **PART_ADDED** — Part added via construction without creating a new vessel. Ghost starts rendering this part mesh after this UT.
- **TIME_JUMP** — Discontinuity from a relative-state time jump (see Section 14.5). Stores pre-jump and post-jump state vectors (position, velocity, UT) so playback can handle the gap as a visual cut.

4.9 Merge Events

When two vessels merge (docking, boarding, claw attachment), both parent segments end and one child segment begins. The child inherits all controllers from both parents.

Merging with a pre-existing persistent vessel (e.g., docking to a station from a previous recording): The current recording's segment ends with a merge event. The `targetVesselId` field links the recording tree to the persistent game state. The combined vessel is a new segment in the current recording's tree. The historical recording tree of the station is NOT modified — it is immutable.

4.10 Example DAG

A crewed lander with a probe-controlled transfer stage:

```
LAUNCH (capsule + transfer + booster)
|
+- SPLIT: staging
|   +- [booster] -- no controller -- DEBRIS
|   +- [capsule + transfer] -- controller: capsule crew
|       |
|       +- SPLIT: undock
|           |   +- [capsule/lander] -- controller: crew
|           |       |   +- SPLIT: EVA
|           |       |       |   +- [kerbal] -- controller: self
|           |       |       |       |   +- MERGE: board
|           |       |       |       |   +- [lander] -- controller: remaining crew
|           |       |       |       |   +- MERGE: board
|           |       |       |       +- [reunified lander]
|           |       |       +- MERGE: dock with transfer
|           |       +- [transfer] -- controller: probe core
|           |       +- MERGE: dock (lander returns)
|           +- [reunified vessel]
|               +- SPLIT: decouple for reentry
|                   |   +- [capsule] -- RECOVERED
|                   |   +- [transfer] -- DESTROYED
```

5. Multi-Vessel Recording Sessions

5.1 The Problem

KSP gameplay frequently involves multiple vessels that are simultaneously relevant: rendezvous, base construction, orbital assembly. The player switches control between vessels, and the recording must capture all of them at appropriate fidelity so that playback looks correct regardless of camera position.

5.2 Recording Session

A recording session is scoped to a single RecordingTree. During a session, the recorder maintains:

- 1. **The focus track:** Full recording (source=Active) of whichever vessel the player is currently controlling.
- 2. **Background physics tracks:** Reduced-rate trajectory sampling (source=Background) + all discrete part events for all other vessels within the physics bubble (~2.3km). Sample rate varies by proximity: <200m: 5Hz, 200m-1km: 2Hz, 1km-2.3km: 0.5Hz.
- 3. **Orbital checkpoints:** Keplerian element snapshots (source=Checkpoint) captured automatically at on-rails transitions, time warp boundaries, and scene changes.

5.3 Data Model

The recording session uses the data structures defined in Section 4.3. In summary: the RecordingTree holds a dictionary of Recordings (one per vessel), connected by BranchPoints. Each Recording's TrackSections list contains typed trajectory chunks tagged with a source (Active, Background, or Checkpoint) for diagnostics. The merge algorithm (Section 9) produces a single merged Recording per vessel with non-overlapping TrackSections selected by highest-fidelity-wins.

5.4 Focus Switching

When the player switches focus from vessel A to vessel B:

- 1. Vessel A's active recording transitions to background (or orbital checkpoint if going on rails).
- 2. Vessel B's background recording transitions to active.
- 3. The tree's ActiveRecordingId changes.

The focus history is implicit in the sequence of ActiveRecordingId changes — no explicit focus log is stored.

5.5 Scene Changes

When the player leaves the flight scene, all vessels go on rails and orbital/surface checkpoints are captured automatically. When the player returns, new checkpoints are captured and physics sampling resumes. The absence of trajectory points between checkpoints IS the gap, and playback propagates from checkpoints during these intervals.

5.6 Time Warp Boundaries

Every time the player enters or exits time warp, Parsek snapshots the orbital elements of all vessels involved in the current recording session. This creates clean reference points for orbital propagation during playback and prevents drift accumulation over long time periods.

6. Segment Taxonomy

Every vessel segment carries two classification axes that are independent of each other.

6.1 Axis 1: Environment

Determined automatically at recording time by vessel state. Changes trigger track section boundaries.

Environment	Trigger	Sample Rate	Notes
ATMOSPHERIC	Below atmosphere ceiling	High (~50 Hz physics tick)	Chaotic trajectory (drag, lift, heating). No analytical reconstruction possible.

EXO_PROPULSIVE	Above atmosphere AND engine producing thrust	High (~50 Hz physics tick)	Thrust is player-controlled, trajectory not Keplerian.
EXO_BALLISTIC	Above atmosphere AND no engines producing thrust	Low (orbital checkpoints)	Keplerian, analytically propagable.
SURFACE_MOBILE	Landed/splashed AND ground speed > 0.1 m/s	Medium (~5-10 Hz)	Rover driving, vessel repositioning.
SURFACE_STATIONARY	Landed/splashed AND ground speed < 0.1 m/s for >3s	Minimal (~0.1 Hz or checkpoint)	Parked vessel, base module. Body-fixed coordinates.

Transitions use hysteresis: 1 second for thrust toggle (prevents rapid toggling during pulsed burns), 3 seconds for surface speed threshold.

6.2 Axis 2: Reference Frame

Reference Frame	When Used	Position Data	Playback
ABSOLUTE	Default for active flight	Body-fixed coordinates (body, lat, lon, alt, attitude)	Ghost placed at recorded coordinates.
RELATIVE	Within physics bubble of a pre-existing persistent vessel	Offset from anchor vessel (dx, dy, dz in anchor's frame)	Ghost position computed from anchor vessel's current position. Tracks anchor precisely.
ORBITAL_CHECKPOINT	On-rails coasting, no physics	Keplerian elements at discrete timestamps	Ghost position computed by Kepler propagation to current UT.

Docking approach rule: Any approach within docking range (~200m) of a pre-existing vessel uses RELATIVE reference frame to ensure positional accuracy for visually correct docking playback.

6.3 Combined Example

A cargo delivery to a Mun base — one VesselSegment (no staging or docking), nine TrackSections:

Recording: "Cargo Run Alpha" (single VesselSegment, 9 TrackSections)	
Track 1: [ATMOSPHERIC + ABSOLUTE]	Kerbin launch through atmosphere. ~50 Hz.
Track 2: [EXO_PROPULSIVE + ABSOLUTE]	Gravity turn completion and orbital insertion.
Track 3: [EXO_BALLISTIC + ORBITAL_CKPT]	Kerbin parking orbit coast. Checkpoint per orbit.
Track 4: [EXO_PROPULSIVE + ABSOLUTE]	Transfer burn to Mun.
Track 5: [EXO_BALLISTIC + ORBITAL_CKPT]	Transfer coast.
Track 6: [EXO_PROPULSIVE + ABSOLUTE]	Munar orbit insertion and deorbit.
Track 7: [EXO_PROPULSIVE + ABSOLUTE]	Powered descent. ~50 Hz.
Track 8: [EXO_PROPULSIVE + RELATIVE]	Final approach within physics bubble of base.
Track 9: [SURFACE_STATIONARY + RELATIVE]	Landed near base. Minimal sampling.

7. Background Vessel Recording

7.1 Principle

When the player controls one vessel, all other vessels within the physics bubble (~2.3km) are actively simulated by KSP's physics engine. Parsek records these background vessels to ensure playback looks correct regardless of which vessel the camera follows.

7.2 Background Trajectory Recording

Background vessels are sampled at reduced rate compared to the focused vessel:

Distance to focused vessel	Sample Rate
< 200m (docking range)	~5 Hz
200m - 1km	~2 Hz
1km - 2.3km (physics edge)	~0.5 Hz

7.3 Background Part Event Recording

All discrete (visual) part events are recorded for ALL vessels in the physics bubble: engine ignition/shutdown, staging, decoupling, parachutes, landing gear, solar panels, antennas, cargo bays, fairings, lights, RCS, EVA, etc.

High-frequency continuous events (engine gimbal angles, control surface deflections, throttle) are captured ONLY for the focused vessel.

7.4 Structural Events for All Physics-Bubble Vessels

Events that affect the DAG structure (staging, decoupling, docking, undocking) are captured for all vessels in the physics bubble. A background vessel that stages while the player is focused elsewhere produces a proper SPLIT event in the recording tree. Debris children from intentional splits get a TTL: recording stops after 30 seconds, or on crash/destruction, or when leaving the physics bubble. Rapid-fire crash fragments are coalesced into BREAKUP events and not individually tracked.

8. Orbital Checkpoint System

8.1 Purpose

For vessels on rails (outside the physics bubble, during time warp, or across scene changes), full trajectory recording is impossible and unnecessary. Parsek records Keplerian orbital elements at discrete points and propagates analytically between them.

8.2 Checkpoint Data

Checkpoint data is stored as `OrbitSegment` and `SurfacePosition` values (defined in Section 4.3). In summary:

```
OrbitSegment (used for orbital checkpoints)
  startUT, endUT, inclination, eccentricity, semiMajorAxis,
  longitudeOfAscendingNode, argumentOfPeriapsis, meanAnomalyAtEpoch,
  epoch, bodyName, orbitalFrameRotation, angularVelocity
```

SurfaceCheckpoint

ut, body, latitude, longitude, altitude, heading

8.3 When Checkpoints Are Captured

- Session start (all on-rails vessels)
- Focus switch (vessel going on rails)
- Time warp enter/exit (all session vessels)
- Scene change (all session vessels)
- SOI transition
- Periodic safety net: every 3-5 orbits for long coasts

8.4 Playback from Checkpoints

The ghost's position between checkpoints is computed by Keplerian propagation from the nearest prior checkpoint. If a ghost's propagated orbit intersects terrain during a gap, the ghost follows the orbit into the ground — this accurately represents what KSP computed.

9. Multi-Vessel Merge

9.1 The Problem

A recording session involving multiple vessels produces overlapping trajectory data. This must be merged into one continuous track per vessel for the committed timeline.

9.2 Merge Rule: Highest Fidelity Wins

For any vessel at any moment in time, use the best available data source:

Priority 1: Active focus recording (full trajectory + events)

Priority 2: Background physics recording from nearest focused vessel

Priority 3: Orbital checkpoint propagation

9.3 Merge Procedure

For each vessel: collect all data sources, select highest-priority source at each UT interval, stitch into one continuous track. Source-switch boundaries use snap-switch (no interpolation) — a 0.5s crossfade at orbital velocities (2200 m/s in LKO) would create 1100m of fake trajectory. The position discontinuity magnitude is logged at each boundary.

9.4 No Circular References

At any given UT, exactly one vessel is the active focus (the scene origin). All other positions derive from that perspective or from orbital mechanics. There is no circularity because focus is a total order.

9.5 Committed Timeline Output

After merge, each vessel has a single clean Recording with non-overlapping TrackSections covering the full session, each tagged with source (Active/Background/Checkpoint) for diagnostics. This is what gets stored in the timeline and used for all subsequent playback.

9A. Chain Segmentation and Recording Optimizer

Section 4.4 defines segment boundaries within the recording tree (structural splits/merges only). This section covers the **chain segmentation** system and the **RecordingOptimizer** that merges or splits committed chain segments at save/load time.

Note (v0.8.0, T56): Chain mode (standalone single-vessel recordings with eager boundary splitting) was removed. All recordings now use tree mode. The boundary detection code (9A.2) is retained in `FlightRecorder` but its flags are always suppressed because `activeTree` is always non-null. Chain segments are now produced exclusively by the `RecordingOptimizer`'s post-commit split pass (9A.5). Sections 9A.1--9A.4 describe the original chain mode architecture for historical reference; section 9A.5 (the optimizer) remains current.

9A.1 Motivation (historical)

Chain mode was removed in v0.8.0 (T56). This section describes the original motivation for chain segmentation, which is now handled entirely by the `RecordingOptimizer`'s post-commit split pass (Section 9A.5).

Chain recordings (the former single-vessel recording mode) produced one Recording per flight. A long flight that traversed multiple environments (atmosphere → vacuum → different SOI → landing) resulted in a single monolithic segment. This made it impossible for the player to selectively loop or configure playback for individual flight phases (e.g., loop just the atmospheric ascent, or hide the boring transfer coast).

Chain segmentation automatically split the recording at environment boundaries so each flight phase became a separate, independently configurable Recording linked by a shared `ChainId`.

Tree recordings (now the only mode) do not use eager chain segmentation — they keep all data in a single Recording with multiple `TrackSections`. Environment changes are tracked internally via the `TrackSection` `environment` field and the session merger (Section 9) handles fidelity resolution. The `RecordingOptimizer`'s split pass (9A.5) produces chain segments post-commit.

9A.2 Chain Segment Boundary Conditions

Seven conditions trigger a chain segment commit-and-restart. The first three are environment boundaries; the rest are vessel lifecycle events.

A. Atmosphere Boundary (atmospheric bodies only)

Detected per-frame by `FlightRecorder.CheckAtmosphereBoundary()`. Fires when the vessel crosses the body's `atmosphereDepth` altitude.

Hysteresis (prevents rapid toggling near the boundary):

- **Time:** 3.0 seconds sustained on the new side
- **Distance:** 1,000 meters past the boundary

Both conditions must be met simultaneously. If the vessel drifts back before both thresholds are met, the pending detection resets.

Phase tags: Outgoing segment tagged `"atmo"` (was in atmosphere) or `"exo"` (was above atmosphere).

B. Altitude Boundary (airless bodies only)

Detected per-frame by `FlightRecorder.CheckAltitudeBoundary()`. Only active when the body has no atmosphere. Fires when the vessel crosses the **approach altitude threshold**.

Threshold computation (`FlightRecorder.ComputeApproachAltitude`):

1. **Primary:** KSP's `timewarpAltitudeLimits[4]` (the 100x warp limit) — this is KSP's own definition of "close enough that fast warp is dangerous" and adapts to modded planets automatically.
2. **Fallback:** `body.Radius * 0.15`, clamped to `[5,000m, 200,000m]`.

Hysteresis:

- **Time:** 3.0 seconds sustained
- **Distance:** `max(1000, threshold * 0.02)` meters past boundary

Phase tags: "approach" (descended below threshold) or "exo" (ascended above).

C. SOI Change

Detected by `FlightRecorder.OnSoiChange()` when a vessel transitions between celestial bodies' spheres of influence. The current segment is committed immediately (no hysteresis — SOI changes are discrete events). Phase tag is derived from the **departing** body's environment at time of exit.

D. EVA Exit (Vessel → EVA)

When a Kerbal goes EVA, the vessel recording commits. The vessel continues being tracked via **adaptive continuation sampling** (`ChainSegmentManager.SampleContinuationVessel()`) — samples are taken on whichever triggers first: time interval (3.0s default), velocity direction change (2.0°), or speed change (5%). A new Recording starts on the EVA Kerbal.

E. EVA Boarding (EVA → Vessel)

When a Kerbal boards a vessel, the EVA segment commits and a new vessel recording begins.

F. Dock/Undock

Handled by `ParsekFlight.HandleDockUndockCommitRestart()` with four sub-scenarios (initiator dock, target dock, undock-stay, undock-switch). Each causes the current segment to commit. On undock, the sibling vessel gets a **ghost-only continuation** (`ChainBranch=1`) — it is adaptively sampled but never spawns as a real vessel at playback time.

G. Vessel Switch During Active Chain

If the player switches vessels while a chain is recording, the entire chain terminates (`HandleVesselSwitchChainTermination`). No continuation is possible because the focused vessel has changed.

Not a Boundary

Staging and part joint breaks are recorded as `PartEvents` within the continuing segment. They do **not** cause chain splits.

9A.3 Tree Mode Suppression

When recording in tree mode (`activeTree != null`), atmosphere, altitude, and SOI boundary splits are **suppressed**. The boundary flags on the FlightRecorder are cleared (`"Boundary flags cleared (tree mode bypass)"`), and the data stays in the single tree Recording. The environment changes are captured as `TrackSection` boundaries within that Recording instead.

This is because tree recordings use the session merger (Section 9) for multi-source data and the RecordingOptimizer's split pass (Section 9A.5) post-hoc if the user later wants to break them apart.

9A.4 Chain Structure

Each chain segment is a separate `Recording` linked by:

- `ChainId` — shared GUID identifying the chain
- `ChainIndex` — 0-based sequential position (reindexed by `StartUT` after optimizer passes)
- `ChainBranch` — 0 = primary (playable, spawnable), >0 = parallel ghost-only (e.g., undock continuation)

The **boundary anchor** ensures seamless stitching: the first `TrajectoryPoint` of each new segment is copied from the previous segment's last point, guaranteeing zero positional discontinuity at chain boundaries.

Mid-chain segments have their `VesselSnapshot` nulled (ghost-only playback) — only the final segment in the chain retains the full vessel snapshot for spawn-at-end.

9A.5 Recording Optimizer

`RecordingOptimizer.cs` contains pure static functions that merge or split committed chain segments at save/load time. This is a **post-hoc cleanup pass** — it operates on already-committed `Recordings`.

Auto-Merge: `CanAutoMerge(a, b)`

Two consecutive chain segments can be merged if **all** of these conditions hold:

1. Neither is null
2. Same `ChainId` (non-empty)
3. Both have valid `ChainIndex` (≥ 0)
4. Consecutive: `b.ChainIndex == a.ChainIndex + 1`
5. Both on primary branch (`ChainBranch == 0`)
6. No `ChildBranchPointId` on A (no branch point between them)
7. Same `SegmentPhase`
8. Same `SegmentBodyName`
9. Neither has ghosting-trigger events (checked via `GhostingTriggerClassifier`)
10. Neither has `LoopPlayback` enabled
11. Both have `PlaybackEnabled == true`
12. Neither is `Hidden`
13. Both have default `LoopIntervalSeconds` (10.0)
14. Both have default `LoopAnchorVesselId` (0)
15. Same `RecordingGroups` (ordered list equality)

The purpose: if two consecutive segments have the same environment, same body, and neither has been customized by the user, they are redundant splits and should be re-merged into one.

Merge operation (`MergeInto`): Points concatenated, events merged and re-sorted by UT, `TrackSections` concatenated, `VesselSnapshot` and `TerminalState` inherited from the absorbed (later) segment. Ghost geometry cache invalidated on the result.

Auto-Split: `CanAutoSplit(rec, sectionIndex)`

A single `Recording` can be split at a `TrackSection` boundary where the environment changes, if:

1. `Recording` has ≥ 2 `TrackSections`
2. No ghosting-trigger events anywhere in the recording
3. Both resulting halves are ≥ 5 **seconds** long

Split operation (`SplitAtSection`): Points partitioned by UT at the section boundary. Events partitioned (backward loop to avoid index shifting). `TrackSections` split at the section index. `GhostVisualSnapshot` cloned to both halves. Each half tagged with `SegmentPhase` derived from its first section's environment via `EnvironmentToPhase` :

SegmentEnvironment	Phase
Atmospheric	"atmo"
SurfaceMobile	"surface"
SurfaceStationary	"surface"
All others	"exo"

Discovery Passes

FindMergeCandidates : Groups committed recordings by `ChainId` , sorts each group by `ChainIndex` , tests all consecutive pairs with `CanAutoMerge` .

FindSplitCandidates : Scans each committed recording's `TrackSections` for adjacent sections with **different environments**. Tests each boundary with `CanAutoSplit` . Finds **at most one split per recording per pass** — the caller re-scans after each split because indices shift.

FindSplitCandidatesForOptimizer : Same as `FindSplitCandidates` but uses `CanAutoSplitIgnoringGhostTriggers` — does not require absence of ghosting-trigger events (engine ignitions, RCS activation, etc.). This is correct for the optimizer because both halves inherit the `GhostVisualSnapshot` and part events are correctly partitioned by `SplitAtSection` . The ghost rendering system handles part events over time. The conservative `HasGhostingTriggerEvents` check remains on `CanAutoSplit` for other callers (ghost chain walker).

The optimizer is wired into save/load (`ParsekScenario`), running merge candidates first (to clean up redundant same-environment boundaries), then split candidates (to break multi-environment recordings into per-phase segments with individual loop toggles). Split recordings share a `ChainId` for UI grouping — they appear as expandable chain blocks in the recordings window.

9A.6 Chain Segments Within Tree Recordings

Historical note: Prior to v0.8.0 (T56), chain mode and tree mode were two separate recording strategies. Chain mode split eagerly at environment boundaries during recording; tree mode deferred splitting to the optimizer. T56 removed chain mode entirely — all recordings now use tree mode. The description below reflects the current (tree-only) architecture.

All recordings produce `Recording` objects (same class, no subclasses) with the same playback-relevant fields: `TrackSections` , `Points` , `OrbitSegments` , `SegmentPhase` , `SegmentBodyName` , `GhostVisualSnapshot` . Environment changes within a recording are tracked as `TrackSection` boundaries. The `SessionMerger` resolves overlapping data streams (active/background/checkpoint sources) by fidelity priority.

After commit, the `RecordingOptimizer` split pass (Section 9A.5) breaks multi-environment tree recordings into per-phase chain segments — separate `Recording` entries linked by `ChainId` , each with its own loop toggle. Split recordings carry both `TreeId` (for tree-level resource tracking) and `ChainId` (for UI grouping as chain blocks).

The `GhostPlaybackEngine` is completely mode-agnostic — it receives `IPlaybackTrajectory` interface references and has zero knowledge of chains, trees, or policy. This clean abstraction boundary means the engine could be extracted as a standalone library.

10. Looped Recordings

10.1 Principle

Looped recording segments are anchored to a real persistent vessel (station, base, etc.). Parsek never modifies the real vessel's state — it only reads its position to compute ghost placement.

10.2 Lifecycle

```
Real vessel loads into physics range (or visual range)
-> Check: any looped recordings anchored to this vessel?
-> Compute where each loop is in its cycle based on elapsed UT
-> Spawn ghost(s) at the correct loop phase
-> Ghosts update position relative to real vessel's actual current position
```

```
Real vessel unloads
-> Destroy all ghosts anchored to it
-> Store each ghost's current loop phase for resumption
```

10.3 Segment Looping

The player marks which segments to loop. Multiple segments can be looped together as a sequence (e.g., powered descent + approach = complete arrival visual). The loop restarts from the first selected segment after the last completes.

10.4 Looped Segments Are Always Relative

When a looped segment plays back, it uses the real anchor vessel's current position, not the historical position from recording time. This means zero positional drift regardless of elapsed time. If the anchor vessel no longer exists, the loop is marked as broken.

10.5 Validation on Spawn

Before spawning a looped ghost, Parsek validates: does the anchor vessel still exist? Is it on the expected body? Is the docking port still available? If validation fails, the loop is marked as broken in the recordings manager.

10.6 Per-Phase Looping

All recordings support per-phase looping. The optimizer split pass (Section 9A.5) breaks multi-environment tree recordings into per-phase chain segments post-commit. The result: separate Recording entries per flight phase, each with its own `LoopPlayback` toggle in the recordings window.

Auto loop range: When a recording's loop toggle is enabled, `ComputeAutoLoopRange` trims boring bookends (`ExoBallistic` orbital coasts, `SurfaceStationary` idle) to auto-select the visually interesting portion. This narrows the loop to the action phase without user intervention.

Loop range fields: The Recording carries optional `LoopStartUT` and `LoopEndUT` fields (default `double.NaN` = loop entire recording). When set, the engine's loop math (`TryComputeLoopPlaybackUT`) uses these bounds instead of `StartUT` / `EndUT` . The auto loop range sets these automatically; they are also available for manual customization.

Architecture: The `LoopStartUT` / `LoopEndUT` fields are exposed via `IPlaybackTrajectory` .

`EffectiveLoopStartUT` / `EffectiveLoopEndUT` static helpers provide cross-validated bounds with inverted-range fallback. No modification to the `TrackSection` struct or the `GhostPlaybackEngine` 's core architecture — only the loop UT computation narrows its range.

11. Rendering Zones

11.1 The Floating Point Problem

KSP uses Unity's single-precision float coordinate system with the scene origin pinned to the active vessel. At large distances, positional precision degrades:

- At 100km: ~0.01m precision (acceptable)
- At 500km: ~0.05m precision (visible jitter)
- At 1000km+: severe jitter, unusable

11.2 Zone Definitions

Zone	Range	Playback	Looped Ghosts
Zone 1: Physics Bubble	0-2.3 km	Full mesh, part event replay, relative segments active	Full fidelity
Zone 2: Visual Range	2.3-120 km	Ghost mesh, orbital propagation, no part events	Spawned if <50km, simplified
Zone 3: Beyond Visual	120 km+	No ghost rendered, position tracked for map view only	Not rendered

11.3 Ghost Rendering Rules

- Zone 2 to Zone 3 (outward): mesh disappears.
- Zone 3 to Zone 2 (inward): mesh appears with short fade-in.
- Zone 2 to Zone 1 (inward): switch from orbital propagation to physics-bubble trajectory data. Part events begin.
- Zone 1 to Zone 2 (outward): switch from trajectory to orbital propagation. Part events stop.

11.4 Soft Cap System

Rather than hard limits, configurable soft caps degrade gracefully:

Condition	Action
Zone 1 ghosts > 8	Reduce background ghost fidelity
Zone 1 ghosts > 15	Despawn lowest-priority ghosts (oldest loops first)
Zone 2 ghosts > 20	Reduce to orbit-line-only

11.5 Map View

In map view, KSP uses double-precision coordinates. Parsek can display ghost positions as icons on the map regardless of distance, without jitter. Ghost orbital paths can be drawn on the map for the full duration of their recording.

12. Interaction with Pre-Existing Persistent Vessels

12.1 Core Rule

Parsek never modifies a real vessel's state directly. The only objects Parsek creates and controls are ghost vessels (raw Unity GameObjects) for playback, and replacement real vessels spawned at ghost chain tips (see Section 12.5).

When Parsek needs to prevent the player from interacting with a vessel during a ghost chain window, it **hides** the real vessel (despawning it and replacing it with a ghost) and **spawns a replacement** at the chain tip. It never edits the vessel's orbit, parts, resources, or crew.

12.2 Background Tracks of Existing Vessels

When a pre-existing vessel appears in the physics bubble during recording, Parsek records its trajectory as background data. This serves as anchor reference data for relative-frame positioning, fallback ghost data if the vessel no longer exists at playback time, and validation data to check the vessel is approximately where expected.

If the real vessel IS present during playback, Parsek does NOT spawn a ghost for it. The real vessel serves as its own visual presence.

12.3 Merge Events with Existing Vessels

When a recording's vessel docks to a pre-existing vessel, the recording tree notes the merge with a `targetVesselId` . The ghost chain model (Section 12.5) resolves how this interaction is replayed when the player rewinds.

12.4 The Vessel Interaction Paradox

When a committed recording includes a physical interaction with a pre-existing vessel (e.g., docking to a station), a dependency is created: that vessel must be in a specific state when the ghost interaction occurs during playback. If the player rewinds and modifies the vessel before the recorded interaction, the timeline breaks.

The Scenario (Three Timeline Runs):

Setup: Station S exists in 100km Kerbin orbit (launched in earlier recording R0). S has one docking port and one probe core.

Run 1 — Normal play. The player launches vessel A and docks it to S, then commits this as recording R1.

UT	Station S	Vessel A	What happens
0	real, bare	—	S orbiting from R0
1000	real, bare	real	A launches, R1 recording starts
1600	real, combined S+A	part of S+A	A docks to S, R1 recording ends

R1 is committed. It covers UT 1000-1600 and contains a MERGE event at UT 1600 targeting S.

Run 2 — Rewind to UT 500. R1 is committed and claims S via the merge event. S becomes a ghost until R1 resolves.

UT	Station S	Vessel A	What happens
500	ghost, bare	—	Quicksave loaded, S is ghost because R1 claims it
1000	ghost, bare	ghost	R1 playback starts, ghost-A appears
1600	real, combined S+A	part of S+A	R1 playback completes, real S+A spawns

This works. S is untouchable as a ghost.

Run 3 — Rewind to UT 500 again (the paradox scenario). R1 is still committed. The player launches a new vessel B at UT 800 and wants to dock B to S at UT 1200 — before R1 completes.

UT	Station S	Vessel A	Vessel B	What happens
500	ghost, bare	—	—	Quicksave loaded, S still ghost
800	ghost, bare	—	real	B launches
1000	ghost, bare	ghost	real	R1 playback starts
1200	ghost, bare	ghost	real, wants to dock	B cannot dock — S is a ghost
1600	real, combined S+A	part of S+A	real	R1 completes, real S+A spawns. B can now dock.

B cannot interact with S before UT 1600. After UT 1600, S+A exists as real and B can dock to it. The player simply time-warps past the ghost window — zero gameplay cost in stock KSP (no time-dependent consumables). Alternatively, the player uses the relative-state time jump (Section 14.5) to skip past the ghost window while preserving rendezvous alignment.

12.5 The Ghost Chain Rule

When a committed recording contains a physical interaction with a pre-existing vessel, that vessel becomes a ghost from the moment the rewind quicksave is loaded until the recording completes and the final-form vessel spawns as real. The ghost window starts at whatever UT the rewind loads — not necessarily UT=0.

If multiple committed recordings interact with the same vessel lineage, the ghost chain extends:

Recording	Event	Ghost chain state
R1	A docks to S at UT 1600	bare-S ghost -> S+A ghost
R2	B docks to S+A at UT 2000	S+A ghost -> S+A+B ghost
Spawn	—	real S+A+B spawns at UT 2000

The real vessel materializes only at the tip of the chain — after the last committed recording that touches the vessel lineage completes. Parsek walks the full chain of committed recordings that reference a vessel's lineage to determine the spawn point.

Chain walker algorithm:

```
function ComputeAllGhostChains(committedTrees, rewindUT):
    claims = {} // map of vesselPID -> list of {treeId, recordingId, branchPointId, ut, type}

    // Step 1: Scan all committed trees for vessel-claiming events
    for each tree in committedTrees:
        for each branchPoint in tree.BranchPoints:
            if branchPoint.Type in {Dock, Board, Undock, EVA, JointBreak}
               and branchPoint.TargetVesselPersistentId != 0:
                claims[branchPoint.TargetVesselPersistentId].add({
                    treeId: tree.Id, ut: branchPoint.UT, type: "BRANCH_POINT"
                })
```

```

// Also scan background recordings for ghosting-trigger PartEvents
for each recording in tree.Recordings where recording is background:
  if recording has any PartEvent with ghosting-trigger type (not LightOn/LightOff):
    claims[recording.VesselPersistentId].add({
      treeId: tree.Id, ut: event.ut, type: "BACKGROUND_EVENT"
    })

// Step 2: Build chains, sorted by UT
chains = {} // map of vesselPID -> GhostChain
for each (vesselPID, claimList) in claims:
  sort claimList by UT ascending
  chain = new GhostChain(originalVesselPid: vesselPID, ghostStartUT: rewindUT)
  chain.links = claimList
  chain.tipRecording = find the Recording at the last claim's endpoint
  chain.spawnUT = chain.tipRecording.EndUT
  chain.isTerminated = chain.tipRecording.TerminalStateValue in {DESTROYED, RECOVERED}
  chains[vesselPID] = chain

// Step 3: Cross-tree linking – extend chains across trees
for each chain in chains:
  tipRecording = chain.tipRecording
  tipVesselPID = tipRecording.VesselPersistentId // PID of vessel at chain tip
  // Check if any OTHER chain claims this PID
  if tipVesselPID in claims and tipVesselPID != chain.originalVesselPid:
    // Merge into the existing chain for tipVesselPID
    extend that chain with this chain's links
  // Also check: does any BranchPoint in another tree have
  // TargetVesselPersistentId == tipVesselPID?
  // If yes, this chain extends through that tree.

// Step 4: Compute final spawn points
for each chain in chains:
  walk to the furthest link -> that Recording's endpoint is the spawn point
  chain.spawnUT = furthest link's Recording EndUT
  chain.tipRecording = furthest link's Recording

return chains

function IsIntermediateChainLink(chains, recording):
  // True if this recording's EndUT matches a non-final link in any chain
  for each chain in chains:
    if recording's EndUT matches a link that is NOT the chain tip: return true
  return false

```

Output: A map of {vesselPID -> GhostChain} where each GhostChain contains: the original vessel PID, the ghost start UT, the ordered list of chain links, the tip Recording (which provides the spawn snapshot), the spawn UT, and whether the chain terminates (no spawn). This map is recomputed on every rewind and save load from the committed trees — it is never persisted.

12.6 Ghosting Trigger Taxonomy

A committed recording forces ghosting on a pre-existing vessel if and only if the recording contains a recorded event that changes that vessel's physical state.

Physical state changes (TRIGGER ghosting):

Structural:

- Docking to vessel (MERGE)
- Undocking from vessel (SPLIT)
- Part destruction on vessel (collision, explosion, overheat)
- Crew transfer to/from vessel (kerbal = vessel; boarding = merge, EVA = split)
- Claw/AGU grab (structural MERGE)
- EVA construction: part added to or removed from vessel

Orbital / positional:

- Engine burns (changes orbit)
- RCS translation (changes position/orbit)
- Staging (fires decouplers, changes mass distribution and trajectory)

Part state:

- Deploying/retracting parts (solar panels, radiators, antennas, landing gear)
- Running/collecting science experiments
- Action group triggers that change physical part state
- Resource transfers (fuel, monoprop)

Does NOT trigger ghosting:

Cosmetic: toggling lights, animations without physical effect.

Observation: switching focus to the vessel, switching focus away, camera movement, map view, recording the vessel's background trajectory, SAS mode changes, minor orbital perturbation from physics settling.

The dividing line: Would removing this event from the timeline leave the vessel in a different physical state than if it had been left alone? If yes, the event triggers ghosting. If no, it does not.

12.7 Looped Recording Interactions

Looped recordings that include interactions with pre-existing vessels follow a different rule:

- The **first run** of a looped recording is the real run — it follows the ghost chain logic described above. The vessel spawns as real at completion of the first run.
- **All subsequent loops** are purely visual. Ghosts appear and disappear on the loop cycle. They do not affect gameplay, do not spawn real vessels, and do not extend the ghost chain.

Definition of "first run": The first chronological playback of the recording after its commit point. If the recording was committed at UT=1000, the first playback spanning UT=1000 to 1600 is the real run. If the player rewinds to before UT=1000 and fast-forwards again, the playback starting at UT=1000 is still the first run — "first" is defined by the recording's timeline position, not by how many times the player has rewound. The real run always corresponds to the recording's original UT span. Every subsequent loop iteration is visual only.

12.8 Undocking Scenarios

Undocking follows the same ghost logic as docking. If R1 involves undocking a module from S (a SPLIT event), S's physical state changes. Therefore S is a ghost until R1 completes, at which point S spawns as real in its post-undocking form (fewer parts). The undocked module also spawns as a separate real vessel. Both products of the split

are independently subject to ghost chain rules. If a later committed recording docks something to the undocked module, that module's ghost chain extends independently.

12.9 Edge Cases

12.9.1 — Vessel destruction terminates the chain. If R1 crashes into S and destroys S's controller, the ghost chain terminates with no spawn. Ghost-S plays the crash visually, then disappears. Any surviving piece that has a controller (e.g. a secondary probe core) is a vessel under Parsek's identity model and spawns as a real vessel at its final recorded position. Pieces without a controller are debris and do not spawn. The ghost plays the breakup animation; controller-carrying pieces transition from ghost to real at the chain tip, debris pieces simply vanish.

12.9.2 — Spawn-point collision. At a chain tip, a real vessel spawns at the ghost's position. If the player has parked another vessel at that position, the spawn could cause physics overlap. Resolution: bounding box overlap check at spawn time. See Section 13 (Spawn System) for full details.

Full spawn collision sequence:

Step	What happens
UT approaching chain tip	Warning appears: "S+A spawning in Xs — vessel B is Ym from spawn point"
UT reaches chain tip	Bounding box overlap check. If overlap: spawn blocked, ghost continues on propagated orbit.
Each frame while blocked	Recheck bounding box overlap at ghost's current propagated position. Warning persists.
Overlap clears	Spawn real vessel at ghost's current propagated position. Ghost destroyed. Warning dismissed.

12.9.3 — Independent ghost chains. If R1 docks A to S1 and R2 docks B to S2, both chains are independent. The player must wait for both to resolve before interacting with S1 or S2. The player should be informed on screen that real vessels only spawn at the end of their timeline.

12.9.4 — Recording scope and vessel claiming. A recording is tied to its vessel controller. A recording does NOT claim a pre-existing vessel merely by switching focus to it. Each vessel has its own separate recording timeline. When the player switches control to a different vessel during a recording session, they are switching to that vessel's own recording chain. A recording can only claim a pre-existing vessel through physical interaction (Section 12.6). If the player only observed a vessel without physical interaction, that vessel remains real after rewind.

12.9.5 — Intermediate spawn suppression and ghost visual transition. When a ghost chain has multiple links (bare-S -> S+A -> S+A+B), Parsek suppresses the spawn at intermediate points. At UT 1600, R1 completes and would normally spawn real S+A, but R2 claims S+A's controller (docking B at UT 2000). Parsek detects this: the controller that would spawn is referenced by a later committed recording. The spawn is suppressed, S+A continues as a ghost, and only S+A+B spawns at UT 2000. Rule: before spawning a real vessel at a chain link, check whether any committed recording further down the chain claims that vessel's controller.

Ghost mesh transition at intermediate links: When ghost-S reaches R1's completion UT and the spawn is suppressed, the ghost's visual must change from bare-S to S+A (the post-merge form). The mechanism: destroy the current ghost GameObject (bare-S mesh), create a new ghost GameObject from R1's endpoint VesselSnapshot (which contains the S+A part layout). This is a visual transition only — the ghost remains a ghost. The new ghost continues on the chain using R2's trajectory data.

12.9.6 — Cross-perspective interaction attribution. A recording is of vessel B. During that recording, B collides with station S and breaks off S's solar panel. The multi-vessel merge algorithm already walks background recordings of all vessels in the physics bubble, detects the part destruction event on S, and attributes it to S's own recording timeline. S's timeline now contains a ghosting-trigger event. No special mechanism is needed.

12.9.7 — Single recording with vessel separation. R1 launches vessel A. A separates into A1 and A2. A1 docks to S1, A2 docks to S2. The DAG handles this naturally — the chain walker follows edges through split/merge nodes. S1 and S2 are ghosted independently.

12.9.8 — Vessel recovery terminates the chain. R1 deorbits S and the player recovers it. On rewind, ghost-S plays the deorbit trajectory. At the chain tip, nothing spawns — the vessel was recovered. The game actions system credits recovery funds through the normal earning-action pipeline.

12.9.9 — Spawn queue and time warp. When the player time-warps past a ghost chain tip, spawns are deferred to warp exit. All vessels whose chain tips were crossed during warp spawn as real on warp exit. Spawns within the physics bubble block re-entering time warp until resolved. Spawns outside the physics bubble create a ProtoVessel immediately at the chain tip UT — they appear in the tracking station and map view right away without needing the player's physical presence.

12.9.10 — Self-protecting property. The ghosting model is self-protecting by construction. Once R1 is committed and claims S, S becomes a ghost. A ghost has no physics, no colliders, no docking ports — the player cannot create a second recording R2 that also modifies ghost-S. The only way to interact with S again is to wait for the chain tip spawn. This guarantees all recordings modifying a vessel lineage are naturally ordered — no branching conflicts are possible.

12.9.11 — Asteroids, comets, and debris. Asteroids and comets have no controller — they are debris under Parsek's identity model. However, background recording captures trajectories of all objects in the physics bubble. When a committed recording grabs an asteroid with the AGU (claw), the merge algorithm finds the CLAW merge event and attributes it to the asteroid's timeline. The asteroid is ghosted from rewind until the chain tip. Background recording provides trajectory data; orbital propagation from last known state fills any gaps when the asteroid was outside the physics bubble.

12.9.12 — Surface base position drift. KSP's procedural terrain can shift by a few meters between sessions. The terrain correction system (Section 13.6) handles this via recorded ground clearance, terrain raycast at spawn time, and physics settling.

12.9.13 — KSP load vs Parsek rewind. On any save load, Parsek re-evaluates all ghost chains using the loaded save's committed recording set and current UT. If the loaded save predates a recording's commit, no ghost chain exists. If it postdates the chain tip, the vessel is already real.

12.9.14 — Ghost visual identity. Ghosts must be visually distinguishable from real vessels. Ghost vessels display a floating text label: vessel name, ghost status ("Ghost — spawns at UT=X"), and which recording claims them. EVA kerbals pass through ghosts without interaction — the kerbal clips through the ghost mesh. This is another reason visual distinction matters: without it, the player may EVA a kerbal toward what appears to be a real vessel, only to find the kerbal passes through.

12.10 Fallback: Real Vessel Missing

If a recording references a pre-existing vessel that no longer exists (destroyed, recovered, deorbited since the recording was made):

- For background tracks: spawn a ghost from the background track data as a visual placeholder.
 - For merge events: the ghost reaches the merge point and despawns. The merge target's absence is logged.
 - For looped recordings: the loop is marked as broken (see Section 10.5).
-

13. Spawn System

13.1 Spawn at Recording End

When a recording reaches its end UT, Parsek checks whether to spawn a real vessel:

- If the vessel doesn't exist in the game world, spawn it from the recording's snapshot.
- If it already exists (from the quicksave), skip — the ghost just despawns.
- On revert, spawn state is reconciled after vessel stripping (see Section 13.11).

Looping recordings spawn their vessel at the end of the **first** playthrough. Subsequent loop cycles are visual-only ghosts.

13.2 Chain-Aware Spawn

When ghost chains are active, the spawn logic is extended:

```
Recording reaches EndUT
-> Is this an intermediate chain link?
  -> Yes: suppress spawn, continue ghost chain
  -> No: proceed to spawn
-> Is spawn blocked by collision?
  -> Overlap: block spawn, start ghost extension
  -> Clear: proceed
-> Is this a surface spawn?
  -> Apply terrain correction
-> Spawn vessel (preserving PID for chain-tip spawns)
```

13.3 PID Preservation

For chain-tip spawns, the spawned vessel must preserve the original vessel's persistentId. This is critical for cross-tree chain linking: if R1 docks A to S (PID=100), and the chain-tip spawn produces a vessel with a new PID (e.g. 567), then a later R2 that claims PID=100 cannot link to it. The cross-tree chain breaks silently.

Fix: Chain-tip spawns skip PID regeneration. Since the original vessel was despawned during ghost conversion, the PID is free — there is no collision. With PID preservation: R1's spawn has PID=100 (from snapshot), R2's MERGE has target PID=100, chain walker matches correctly, chain extends.

Normal (non-chain) spawns continue to regenerate PIDs as before.

13.4 Spawn Collision Detection

The spawn system uses two thresholds:

Warning radius (200m): As UT approaches a chain tip, Parsek checks whether any real vessel is within 200m. If yes, a persistent on-screen warning shows distance, vessel name, and countdown. UI-only — does not block spawning.

Bounding box overlap check (spawn blocker): At the chain tip UT, Parsek computes the bounding box of the vessel about to spawn (from its recorded part layout) and checks for geometric overlap with bounding boxes of all real vessels in the loaded physics bubble, plus a small padding margin. If overlap is detected, the spawn is blocked.

13.5 Ghost Extension

When spawn is blocked, the ghost continues past the recording's end time. The recording's final orbital elements are used to propagate the ghost's position via Keplerian orbit math (or surface-stationary persistence). No recorded

trajectory data is needed — the ghost coasts naturally on rails.

Each physics frame while blocked, Parsek rechecks bounding box overlap at the ghost's current propagated position. When overlap clears (the player moves their vessel away), the real vessel spawns at the ghost's current propagated position at the current UT. This is physically correct: the spawned vessel is exactly where a real vessel on that orbit would be at the current time.

Surface case: Same logic but simpler. Ghost stays at surface coordinates. No orbital propagation needed. Player drives their rover away from the base's footprint, spawn fires.

SOI transitions: If the ghost's recorded trajectory crosses SOI boundaries (e.g., Kerbin orbit to Mun orbit), the orbital propagation must switch reference bodies at recorded SOI checkpoint boundaries. The checkpoint system (Section 8.3) captures SOI transitions, providing the orbital elements in the new reference body's frame.

If the player never moves, the ghost persists indefinitely. The warning stays on screen. During time warp both ghost and real vessel are non-physical so no conflict exists, but on warp exit the check runs again.

13.6 Terrain Correction

For surface spawns, KSP's procedural terrain can shift by a few meters between sessions.

Step 1 — Record terrain height. For surface-stationary segment endpoints, the recording stores vessel altitude and terrain height as separate values (the `TerrainHeightAtEnd` field). Ground clearance = recordedAlt - recordedTerrainHeight.

Step 2 — Terrain raycast at spawn. Before spawning, cast a ray downward from above the recorded coordinates to get current terrain height at that lat/lon on that body.

Step 3 — Altitude correction. Spawn altitude = currentTerrainHeight + recordedClearance.

Step 4 — Physics settling. A brief physics settling period (a few frames) where landing legs and wheels interact naturally with terrain. This handles sub-meter errors below raycast resolution.

Ghost extension applies here too. If the bounding box check blocks the spawn, the ghost continues at its surface coordinates. When the spawn eventually fires, the raycast correction uses terrain height at that moment.

13.7 Trajectory Walkback

The standard spawn collision system assumes the player can move to clear the overlap. This fails when the blocking vessel is immovable infrastructure (surface base, ground-anchored station).

Resolution: After a timeout (5 seconds of persistent overlap with no blocking vessel movement), walk backward along the spawning ghost's recorded trajectory frame-by-frame, checking bounding box overlap at each position. Spawn at the latest non-overlapping position. Recompute orbital elements (or surface coordinates) for the new spawn position. The result: the rover materializes a few meters back from where it originally parked — as if the base grew and the parking spot is now occupied.

Fallback: If the entire trajectory overlaps (blocking vessel grew to cover the full approach path), show a manual placement UI within a configurable radius. This should be rare — it requires the blocking vessel to have grown enough to cover the entire approach path.

13.8 Spawn Queue and Time Warp

During time warp, vessel spawning is deferred to a spawn queue. When a recording completes during warp, the spawn is queued. The queue is flushed on warp exit. This prevents physics instability from spawning vessels during warp. Visual FX are suppressed above 10x warp, ghost meshes hidden above 50x warp for performance.

Spawn queue blocks re-entering time warp until all spawns within the current loaded physics bubble are resolved. Spawns outside the physics bubble do not block warp.

13.9 Loaded vs Unloaded Spawning

Loaded spawn (vessel inside physics bubble): Full sequence — ghost replacement, bounding box check, ghost extension if blocked, terrain raycast for surface, physics settling.

Unloaded spawn (vessel outside physics bubble): Create a ProtoVessel entry in the save data at the correct orbital elements or surface coordinates. No Unity objects, no bounding box check. The vessel appears immediately in the tracking station and map view and propagates on rails like any normal unloaded vessel.

13.10 Ghost Conversion

When a vessel must be ghosted:

1. Quicksave loads — real vessel exists.
2. Ghost chain walker identifies the vessel as claimed by a committed recording.
3. Vessel's snapshot is captured (via the existing backup snapshot mechanism).
4. Real vessel is despawned (removed from the game).
5. Ghost GameObject is created from the captured snapshot (using the same visual builder that creates recording ghosts).
6. Ghost follows the background recording trajectory (or orbital propagation when no trajectory data exists).
7. At chain tip: final-form vessel spawns from the tip recording's snapshot (with PID preservation), ghost is destroyed.

Safety: the quicksave is always available as a full backup. Ghost conversion is wrapped in error handling — if it fails at any step, the vessel is left untouched and current behavior applies (no paradox prevention, but no data loss).

13.11 Spawn State Reconciliation After Revert

Spawn tracking uses `SpawnedVesselPersistentId` on each Recording to prevent duplicate spawns — if the PID is non-zero, the spawn system treats the vessel as already spawned and skips it. This creates an edge case on revert:

1. Recording reaches EndUT → vessel spawns → `SpawnedVesselPersistentId = X`
2. Game saves (PID = X persisted in `.sfs`)
3. Player reverts/rewinds
4. OnLoad restores `SpawnedVesselPersistentId = X` from save
5. Vessel stripping removes the spawned vessel (PID = X no longer in `flightState`)
6. PID is still X → dedup check blocks re-spawn permanently

The problem: spawn tracking is restored from the save *before* vessel stripping runs, but stripping removes the vessel the PID points to. The recording thinks it already spawned, but the vessel no longer exists.

Fix: `ReconcileSpawnStateAfterStrip` runs after all vessel strip operations complete. It collects the set of surviving vessel PIDs from `flightState`, then iterates all recordings: any recording whose `SpawnedVesselPersistentId` is non-zero but absent from the surviving set has its spawn tracking reset (`SpawnedVesselPersistentId = 0` , `SpawnNeedsRecovery = false`). The ghost replays normally and re-triggers the spawn at EndUT.

The reconciliation is called at both revert sites in `ParsekScenario` (OnLoad revert path and the rewind path) to cover all vessel stripping scenarios.

14. Rewind and Timeline Operations

14.1 Core Principle

Committed recordings are permanent — they survive rewind. Rewinding moves the player's position on the timeline but does not erase committed recordings. All committed recording ghosts play at their recorded UTs regardless of the player's current position on the timeline.

Each committed recording has an associated quicksave taken at its launch UT. This quicksave captures the game state needed to resume from that point.

Timeline immutability: Individual recordings cannot be deleted after commit. The player's only decision point is at recording end: commit to timeline or discard. The entire timeline can be wiped, but individual recordings cannot be removed. This prevents time paradoxes — deleting a recording mid-timeline could orphan spawned vessels, break chain continuity, or create inconsistent ghost playback.

14.2 Rewind Procedure

The player is at UT=6000 and rewinds to Recording B's launch at UT=2000:

1. Load quicksave at UT=2000.
Vessels spawned by earlier completed recordings ARE in the quicksave.
2. Ghost chain evaluation fires:
 - All committed recordings are scanned for vessel-claiming events.
 - Any claimed pre-existing vessels are ghosted (despawned, replaced by ghost GOs).
 - Chain state is NOT persisted – it is re-derived from committed recordings on every load.
3. Ghost playback resumes from committed recordings:
 - Recordings completed before UT=2000: their spawned vessels are in the quicksave.
 - Recordings in flight at UT=2000: ghosts play at their trajectory positions.
 - Recordings not yet started: no ghosts yet, they appear at their StartUT.
4. Player resumes at UT=2000 with correct state.

14.3 Spawn at Recording End

At each recording's end UT: if the vessel doesn't exist in the game world, spawn it from the recording's snapshot. If it already exists (from the quicksave), skip. On revert, spawn state is reconciled after vessel stripping (Section 13.11) to ensure re-spawn is not blocked by stale PID references.

14.4 Fast-Forward (Time Warp with Ghost Playback)

After rewinding, the player can time warp forward. All committed recording ghosts play at their recorded UTs. The player can drop out of time warp at any point and start flying. Ghosts continue playing around them.

14.5 Relative-State Time Jump

Normal time warp (Section 14.4) is the default way to advance time. The timeline plays out: ghosts appear, play their trajectories, despawn or spawn vessels at their endpoints. The player can fast-forward to any point in the future at any time. For most gameplay, this is sufficient.

The time jump is a specialized mechanism for a specific scenario: **the player has set up a multi-vessel rendezvous approach to a ghosted vessel and needs to skip past its chain tip without losing the alignment.**

The Problem: The player rewinds to T0. Ghost-S is in the physics bubble, chain tip at T1. The player maneuvers vessel A into a rendezvous approach — 80m from ghost-S, velocities matched, docking alignment set up. To interact with S, the player must advance UT past T1. Normal time warp propagates A and ghost-S on their independent Keplerian orbits. Since they are 80m apart, their orbital elements differ slightly. Over the warp interval, differential drift separates them — potentially by hundreds of meters. The carefully set up rendezvous is destroyed.

The Solution: A time jump is a discrete UT skip (not a warp) that advances the game clock while keeping every vessel in the physics bubble at its exact current position. No vessel moves. No intermediate physics simulation occurs. Orbital epochs are adjusted so that Keplerian propagation remains consistent with the new UT.

The Operation:

1. **Snapshot the bubble.** For every object in the loaded physics bubble (ghosts, real vessels, debris), capture:

```
BubbleSnapshot
playerVessel:      active vessel
jumpDelta:         targetUT - currentUT
objects: [
  {
    id:             object identifier
    position:        current absolute position (body-relative)
    velocity:        current absolute velocity
    attitude:        rotation quaternion
    orbitalElements: current elements (to be epoch-shifted)
    isGhost:         bool
    chainTip:        UT of chain tip (if ghost)
  }
]
```

2. **Jump UT.** Set the game clock to the target UT. No physics frames are simulated between T0 and the target UT.
3. **Adjust orbital epochs.** For every vessel in the bubble (including the player's): keep position and velocity vectors unchanged, but recompute orbital elements to be consistent with the new UT at that position. In practice, this means shifting each orbit's mean anomaly at epoch by the jump delta (targetUT - T0). The orbit shapes (SMA, eccentricity, inclination, LAN, argument of periapsis) remain identical. Only the phase reference changes so that Keplerian propagation produces the correct position at the new UT.
4. **Process spawn queue.** For every ghost whose chain tip was crossed during the jump: destroy the ghost, spawn the real vessel at the ghost's current position (which has not moved), apply bounding box overlap check. If overlap, ghost extension or trajectory walkback applies.
5. **Process game actions.** Trigger recalculation for the new UT — science, funds, reputation, kerbals, facilities, contracts. Same as warp exit. (If the game actions system is not yet implemented, this step is skipped with a log warning.)
6. **Resume physics.** The player is at the target UT, at the same position and velocity relative to the now-real vessel, with the same approach geometry.

What changes visually: Planet rotation (Kerbin has rotated to the correct angle for the new UT), sun angle (day/night terminator has moved), star field. All celestial bodies are at their correct positions. These serve as natural visual cues that the jump occurred.

What does NOT change: Vessel position vectors, velocity vectors, relative positions within the bubble, relative velocities, vessel attitudes, approach geometry, orbit shapes.

Surface case: Both vessels surface-fixed. The body rotates with UT, carrying both. Relative positions preserved automatically (surface-fixed coordinates don't change). The only visual change is the sun angle.

Atmospheric case: Epoch-shifting is approximate for atmospheric vessels (trajectory is not Keplerian). Position is preserved exactly, but post-jump orbital elements may not match the atmospheric trajectory that would have occurred. This is acceptable: atmospheric chain tips are rare, relative positions are preserved, and post-jump atmospheric physics resumes immediately.

Looped ghosts during a jump: Looped ghosts that are in the bubble stay at their current positions. Their loop phase advances to match the new UT — orbital epochs are shifted like everything else. They continue their loop cycle from the correct phase. No spawn occurs (loops beyond the first are visual only).

Interaction with active recording: If the player is recording during a time jump, a TIME_JUMP SegmentEvent is stored with pre-jump and post-jump state vectors. Playback handles the discontinuity as a visual cut — no interpolation across the gap.

14.6 Selective Spawning UI

The player does not need to jump to the furthest chain tip. A spawn control panel appears when the player is within the physics bubble of one or more ghosts:

Ghosts in physics bubble:

```
[Sync & Spawn] Station Alpha
  Becomes real at UT=1600 (Recording R1)

[Sync & Spawn] Fuel Depot + Cargo Pod
  Becomes real as combined vessel at UT=3000
  (Chain: R2 -> R3)

[Info] Tanker Ghost
  Loop iteration -- visual only, no spawn
```

Rules:

- Only chain tips where a real vessel actually spawns are offered — not intermediate chain links (Section 12.9.5).
- Linked chains are grouped and shown as a single spawn option with the combined vessel name.
- Loop iterations beyond the first are shown as informational only — no spawn button.
- Each option shows: vessel name, spawn UT, and which recording(s) created the chain.

Spawn selection behavior: When the player selects a spawn option, the target UT is the selected chain tip's UT. Any independent chain tips chronologically before the target are also spawned — a ghost cannot remain ghost past its chain tip. The UI warns: "Also spawns: [vessel name] at UT=[earlier tip]." Ghosts with chain tips after the target UT remain ghost at their current positions.

Chronological constraint: The player cannot jump backward — only forward. The player also cannot jump to a UT before the earliest unresolved chain tip in the bubble if that would require a ghost to exist past its tip.

Chained jumps: The player can perform multiple jumps in sequence. Each jump epoch-shifts whatever is currently in the bubble (including real vessels spawned by previous jumps).

14.7 Time Jump Scenario Walkthroughs

Simple two-vessel rendezvous:

UT	State	Action
T0=500	A: real. Ghost-S: 80m away.	Player sets up docking approach.
—	—	Player selects "Spawn Station Alpha (T1=1600)"
T1=1600	A: real, same position. S: real, still 80m away.	UT jumped. Planet rotated. Nothing in the bubble moved.
T1+	A docks to real S.	Recording R2 starts.

Three vessels, player picks the middle one:

Physics bubble at T0=500: A (real), Ghost-S1 (tip T1=1600), Ghost-S2 (tip T2=2000), Ghost-S3 (tip T3=5000). Player selects "Spawn S2 (T2=2000)." UI warns: "Also spawns: S1 (T1=1600)."

Object	Before jump (T0)	After jump (T2=2000)
A	real	real, same position, epoch-shifted orbit
S1	ghost (tip T1=1600)	real (tip T1 crossed) at same position
S2	ghost (tip T2=2000)	real (tip T2 = target) at same position
S3	ghost (tip T3=5000)	ghost, same position

Player docks to S2. Later, if needed, selects "Spawn S3" for another jump.

Linked chain — player must wait for full chain:

R1 docks X to S at T1. R2 docks Y to S+X at T2. Chain: bare-S -> S+X -> S+X+Y. UI shows only the chain tip:

```
[Sync & Spawn] Station Alpha + X + Y
  Becomes real as combined vessel at UT=2000
  (Chain: R1 -> R2)
```

T1 is NOT offered as a spawn option — intermediate spawn suppression prevents it.

Surface base approach:

Rover A is 50m from ghost-base S on the Mun. Chain tip at T1. Player selects "Spawn Base S." Jump to T1. Both surface-fixed — Mun rotates, sun angle changes, but relative positions identical. Base spawns as real. Rover drives up and docks.

Jump, then rewind: Player jumps to T1, S1 spawns real. Player docks A to S1, commits R2. Player then rewinds to T0. Everything resets: S1 is ghost again (R1 claims it), R2 is committed and plays as ghost. The time jump left no persistent state — standard rewind rules apply.

14.8 Quicksave Pruning

Deferred. All quicksaves are kept. Pruning is purely storage management with no architectural impact. When implemented: manual control (rewind-protected vs archive), auto-pruning policy (keep N most recent), cascading

rewind (if quicksave pruned, rewind to nearest prior quicksave, then fast-forward).

15. Ghost Implementation

15.1 Ghosts Are Purely Visual

Ghosts are playback-only visual objects. They never become real vessels. They never carry enough state to be "promoted" to real vessels. There is no "take control of a ghost" feature — this is explicitly out of scope due to the complexity of reconstructing full vessel state from recorded data mid-flight.

15.2 Raw Unity GameObjects

Ghosts are implemented as raw Unity GameObjects with meshes cloned from part prefabs — NOT as KSP Vessel objects. All visual effects (engine flames, staging, parachutes, solar panels, etc.) are re-implemented by Parsek specifically for ghost playback.

Advantages:

- Ghosts are invisible to KSP's vessel management — no tracking station entries (unless explicitly registered), no persistence overhead, no accidental interaction with game systems.
- No KSP per-vessel overhead (orbit calculation, thermal simulation, resource processing).
- Cleaner separation of concerns.
- Safer — no risk of ghost state leaking into the save file or confusing other mods.

Tradeoffs:

- All visual effects must be manually re-implemented for ghost playback.
- Modded parts with custom shader effects or animations may not replay perfectly.

Performance: Since ghosts are not KSP Vessels, the ghost count limit is driven purely by Unity rendering cost (draw calls, polygon count), not by KSP's vessel management overhead.

15.3 Ghost Interaction Rule

Ghosts do not physically interact with anything:

- No collisions with real vessels, terrain, or other ghosts.
- No resource transfer to or from ghosts.
- No docking with real vessels (ghost docking events are visual only — the ghost approaches and despawns at the merge point).
- The player cannot click on or select ghosts for vessel control.
- EVA kerbals pass through ghosts without interaction.

Ghosts are visible scenery. They make the world feel alive but do not participate in game mechanics.

15.4 Ghost Visual Identity

Ghost vessels display a floating text label: vessel name, ghost status ("Ghost — spawns at UT=X" or "Ghost — loop replay"), and which recording claims them. Labels are visible at all distances within Zone 1 and Zone 2, scaled by distance.

The visual treatment must not obscure the vessel's shape (the player needs to see what vessel it is). The visual treatment should be consistent across all ghost types (chain ghosts, loop ghosts, background trajectory ghosts).

Full visual treatment (transparency, color tint, outline effect) is deferred. The label provides minimum viable distinction.

15.5 Ghost World Presence

Ghosts represent vessels that exist in the world pending chain resolution. They participate selectively in game systems:

Tracking station: Ghosted vessels appear with a distinct icon. The info panel shows: vessel name, ghost status, chain tip UT, claiming recording.

Map view: Ghost orbit line in distinct style (different color or dashed). Vessel marker shows ghost status on hover/click. The player can set a ghosted vessel as a navigation target — approach vectors, closest approach markers, and relative velocity all work normally.

Loaded ghosts (in physics bubble): Unity GameObject for visual rendering, plus a CommNet node registered at the ghost's position with antenna specs from the recording data.

Unloaded ghosts (outside physics bubble): No Unity object. A CommNet node at the orbital-propagated position, plus a tracking station entry and map view marker. This is the minimum representation for a ghost that is far from the player but still participates in the communication network.

Implementation: ProtoVessel-based map presence. Each ghost chain with orbital data gets a lightweight ProtoVessel (single `sensorBarometer` part, `DiscoveryLevels.Owned`). This provides automatic tracking station entries, orbit lines via `OrbitRenderer`, map icons via `MapObject`, and `ITargetable` navigation targeting — all from a single `ProtoVessel.Load()` call. Ghost ProtoVessels are prevented from entering physics simulation via a Harmony prefix on `Vessel.GoOffRails`. They are stripped from saves in `ParsekScenario.OnSave` and reconstructed from recording data on load. A `CommNetVessel.OnStart` patch suppresses the duplicate CommNet node (GhostCommNetRelay handles CommNet separately). Tracking station Fly/Delete/Recover actions are blocked via Harmony patches on `SpaceTracking` methods. All `FlightGlobals.Vessels` iteration sites and vessel GameEvent handlers in Parsek have `GhostMapPresence.IsGhostMapVessel(pid)` guards (27 sites across 9 files). Full design in `docs/dev/done/research/ghost-map-presence-design.md`.

Ghosts are invisible to the recording system. Ghost mesh GameObjects are raw Unity objects (not KSP Vessels). Ghost map ProtoVessels ARE in `FlightGlobals.Vessels`, but every recording system path has an `IsGhostMapVessel` guard that excludes them. The background recorder, flight recorder, spawn collision detector, and all vessel event handlers skip ghost ProtoVessels. If a ghost flies through the physics bubble during an active recording session, the recorder does not see it.

15.6 Ghost CommNet Relay

Antennas on ghosted vessels relay signal, extending communication network coverage. Other real vessels' probe control and science transmission depend on relay paths. A relay constellation placed by a committed recording must provide coverage during the ghost window. The ghost's physical position matters for line-of-sight checks — a relay behind the Mun cannot relay through the Mun.

Implementation: the recording stores antenna data from each vessel's `ModuleDataTransmitter` parts — specifically `antennaPower`, `antennaCombinable`, and `antennaCombinableExponent`. These are captured in `AntennaSpec` entries on the Recording at commit time. Ghost CommNet nodes are registered at ghost positions using these specs. Nodes are updated each frame (loaded: from GO position; unloaded: from orbital propagation). Nodes are removed when the ghost is destroyed or the chain tip spawns. The stock CommNet API (`CommNetNetwork.Instance.CommNet.Add/Remove`) is used directly — no ProtoVessel or Harmony patches required. The implementation was informed by source code analysis of [CommNetManager](#) (confirmed stock API works through its delegate chain) and [RemoteTech](#) (detected at runtime — ghost CommNet registration is skipped when present, since RemoteTech replaces CommNet entirely).

15.7 Passive Resource Generation During Ghost Windows

Passive background processes (science lab processing, ore drilling, ISRU conversion) generate resources on real vessels over time. When a vessel is ghosted, these processes are not running — the ghost is a Unity GameObject, not a KSP Vessel.

This is not a gap. All passive earnings were already captured during the original playthrough and exist as committed game actions on the timeline. The game actions system's recalculation walk processes them regardless of ghost status. Science earned by a lab during the recording's time span is an immutable earning action on the timeline. Funds from contract milestones are on the timeline. Everything is on the timeline.

After the chain tip fires and the real vessel spawns, all passive processes resume normally from that point forward. Any new passive earnings are captured when the player eventually commits.

15.8 Mod Compatibility for Ghost Visuals

Ghosts load part meshes from whatever parts the original vessel used, including modded parts. Base mesh rendering works for any part. Custom visual effects from mods may require explicit support. Unsupported modded visuals degrade gracefully — the mesh is in the right place but the custom animation doesn't play.

16. Data Flow Summary

16.1 Recording Phase

```
Player flies missions, switches vessels, interacts with world
|
+- Focus vessel -> ACTIVE recording (full trajectory + all events)
+- Physics-bubble vessels -> BACKGROUND recording (reduced trajectory + discrete events)
+- On-rails vessels -> ORBITAL CHECKPOINTS at key moments
+- Focus switches -> logged with UT
+- Environment transitions -> trigger track section boundaries
+- Reference frame transitions -> trigger track section boundaries
+- Scene changes / time warp -> checkpoint all session vessels
```

Output: RecordingTree with multi-vessel, multi-track data

16.2 Merge Phase

```
RecordingTree
|
+- For each vessel: select highest-fidelity track at each UT
+- Stitch into continuous single-source tracks
+- Snap-switch at source boundaries (logged discontinuity)
+- Collect DAG events into recording tree
```

Output: Merged Recording per vessel + RecordingTree (DAG)

16.3 Playback Phase

```
Committed timeline with recordings
|
```

```

+- Ghost chain evaluation on scene load:
|   +- Walk committed trees for vessel-claiming events
|   +- Ghost claimed pre-existing vessels
|   +- Compute chain tips and spawn suppression
|
+- For each recording active at current UT:
|   +- Is ghost within Zone 1? -> full mesh + part events
|   +- Is ghost within Zone 2? -> mesh only, orbital propagation
|   +- Is ghost in Zone 3? -> no render, track logically
|
+- For looped recordings:
|   +- Is anchor vessel loaded? -> spawn ghost at current loop phase
|   +- Position relative to anchor's current position
|
+- Chain-tip spawn: when recording ends, check chain status
|   +- Intermediate link: suppress spawn, continue ghost
|   +- Chain tip: spawn real vessel (with PID preservation)
|   +- Apply collision detection, terrain correction, ghost extension
|
+- Part events fire at recorded UT for all visible ghosts

```

17. Recording File Format

17.1 Version Header

Parsek versions its recording sidecar files. Each structural change bumps the version. Any code that reads recordings checks the version first. If unsupported, it fails with a clear message rather than attempting to parse unknown structures.

17.2 Additive Format Evolution

New fields are added, old fields are never renamed or removed. A reader encountering unknown fields safely ignores them and plays back what it understands. This provides forward tolerance: an older Parsek version encountering a newer recording with extra fields can still play back the base trajectory data at reduced fidelity.

17.3 Separation of Concerns

Save file (ScenarioModule – persisted in .sfs):

RecordingTree metadata:

- Id, TreeName, RootRecordingId, ActiveRecordingId
- BranchPoints (full serialization including type-specific metadata)
- Resource deltas (PreTreeFunds/Science/Rep, DeltaFunds/Science/Rep)

Per-Recording metadata:

- RecordingId, TreeId, VesselName, VesselPersistentId
- DAG linkage (ParentRecordingId, ParentBranchPointId, ChildBranchPointId)
- Loop config (LoopPlayback, LoopInterval, LoopAnchorVesselId, LoopAnchorBodyName)
- Spawn state (VesselSpawned, SpawnedVesselPersistentId)
- Terminal state, rewind save references
- Sidecar file path reference

Committed tree list (ordered by commit time)

NOT persisted: ghost chain state (re-derived on every load from committed trees)

NOT persisted: BackgroundMap (runtime only)

Sidecar files (one .prec file per RecordingTree):

- Format version header
- TrajectoryPoints (per Recording, per TrackSection)
- OrbitSegments (per TrackSection, plus legacy flat list)
- PartEvents (per Recording)
- SegmentEvents (per Recording)
- TrackSections (environment, reference frame, source, boundaries)
- VesselSnapshot and GhostVisualSnapshot (ConfigNode blobs)
- AntennaSpecs (per Recording)
- TerrainHeightAtEnd (per Recording)
- ControllerInfo list (per Recording)

This separation keeps saves lightweight and makes file-sharing natural — you share sidecar files, not save games.

18. Performance Budget

18.1 Cost Model

Costs that scale with ghost count:

- Unity draw calls (mesh rendering) — dominant cost for visual ghosts
- Part event processing (checking if any event fires this tick)
- Trajectory interpolation (computing position from recorded frames)

Costs independent of ghost count:

- Recording (only physics bubble matters)
- Timeline management
- Orbital propagation (cheap analytical math)
- Ghost chain evaluation (microseconds for typical tree sizes of 5-20 committed trees)

18.2 Profiling Guidance

The soft cap thresholds (Section 11.4) are estimates. The architecture (zone-based rendering, looped ghost spawn thresholds, anchor-dependent lifecycle) provides all the levers needed to tune performance based on empirical data.

19. Error Recovery

19.1 General Principle

Parsek never crashes or corrupts a save due to bad recording data. Recording failures degrade gracefully: ghosts may be missing or visually imperfect, but gameplay continues.

Ghost chain vessel conversion (Section 13.10) introduces the first modification of real game state. If ghost conversion fails, the vessel is left untouched and current behavior applies (no paradox prevention, but no data loss). The quicksave is always available as a full backup.

19.2 Sidecar File Errors

- **Corrupted sidecar:** Log warning, skip recording, show as "damaged" in recordings manager.
- **Missing sidecar:** Log warning, show as "missing data." Timeline metadata preserved.
- **Unknown version:** Fail with clear message ("requires Parsek X.Y+"). Do not attempt to parse.

- **Partially readable:** Parse what is understood, ignore unknown fields. Reduced fidelity playback.

19.3 DAG Structure Errors

- **Orphaned segment:** Treat as root segment. Ghost plays from its own trajectory data.
- **Empty child segment:** Skip that child. Other children play normally.
- **Missing merge target:** Treat as terminal event. Ghost despawns at merge UT.
- **Circular reference:** Detect during load (topological sort fails). Reject as corrupted.

19.4 Trajectory and Checkpoint Errors

- **NaN/Infinity positions:** Skip affected frames. Use last valid frame.
- **Invalid Keplerian elements:** Skip ghost for the interval. Attempt recovery from next valid checkpoint.
- **Position underground:** Allow it (can be legitimate). Only flag if below body center.
- **Large discontinuity (> 1km):** Log warning. Interpolate over 1 second to smooth.

19.5 Playback and Ghost Errors

- **Anchor vessel missing:** Mark loop as broken. Notify player: "Route to [vessel name] is broken — vessel not found."
- **Anchor on wrong body:** Mark loop as broken.
- **Docking port occupied:** Ghost plays approach but despawns before dock event. Loop continues cycling.
- **Part type missing (mod removed):** Skip visual event for that part. Log on first occurrence per part type.
- **Ghost mesh fails to instantiate:** Skip ghost, retry on next load.
- **Too many ghosts:** Degrade via soft cap rules. Never hard-fail.

19.6 Ghost Chain Errors

Failure	Recovery	Data Loss
Chain walker throws exception	Skip all ghosting. All vessels remain real. No paradox prevention.	None
Snapshot fails	Skip this vessel's ghosting. Vessel remains real.	None
Despawn fails after snapshot	Log critical error. Attempt restore from snapshot. Quicksave as backup.	Possible — vessel may need quicksave reload
Ghost GO creation fails after despawn	Vessel despawned but invisible. Chain walker tracks it as ghosted (spawns at tip). Player sees vessel disappear then reappear.	Visual gap only
Chain-tip spawn fails	Ghost persists past tip. Player reloads quicksave.	Vessel stuck as ghost until reload
Save during partial ghost conversion	On load, chain walker re-evaluates. Both save and quicksave would need to be corrupted for data loss.	Extremely unlikely

Principle: Every failure path defaults to "leave the vessel alone" or "the quicksave has it." The worst case (despawn succeeds but everything else fails) is recoverable via quicksave reload. A clear error message is shown: "Ghost conversion failed for [vessel]. Load your quicksave to recover."

19.7 Persistence Errors

- **Orphaned timeline entries:** Show as "missing data" in manager.
- **Orphaned sidecar files:** Ignore on load, preserve files.
- **Missing quicksave:** Rewind to that recording unavailable. Fall back to nearest prior quicksave.

- **Save during active recording:** Parsek captures session state for resume. If this fails, in-progress recording is lost but committed recordings are unaffected.

20. Backward Compatibility

20.1 Existing Saves

Saves from before the ghost chain system have committed recordings without ghost chain metadata. The chain walker operates on the same recording data that already exists (BranchPoints, PartEvents, background recordings). Existing committed recordings will retroactively trigger ghosting if they contain MERGE/SPLIT events targeting pre-existing vessels. This is correct behavior — the paradox existed before, it just wasn't enforced.

20.2 Recording Format

Ghost chain features add `TerrainHeightAtEnd` (double, NaN default), `AntennaSpec` list (per-vessel antenna data for CommNet), and `SegmentEventType.TimeJump` as additive fields. Old recordings without these fields play back normally (NaN/null/absent = not applicable). Version bump from v6 to v7.

Old recordings (v6 and below) are evaluated by the chain walker based on BranchPoints and PartEvents. No migration needed.

20.3 Ghost Conversion of Quicksave Vessels

On rewind, the quicksave loads vessels that existed at recording start. Claimed vessels are despawned from the quicksave state. The quicksave itself is never modified — it remains a full backup. Loading the quicksave directly (bypassing Parsek rewind) restores all vessels with no ghosting.

Ghost chain state is not persisted — it is re-derived from committed recordings on every load. This guarantees deterministic behavior: the same committed recordings + current UT always produce the same chains.

21. Implementation Status

21.1 Completed Phases (1-5)

The recording system core is fully implemented:

- **Phase 1:** Segment boundary rule, SegmentEvents, crash coalescing, environment taxonomy, reference frames, orbital checkpoints, background part events.
- **Phase 2:** Multi-vessel sessions, focus switching, background trajectory recording with proximity-based sample rate, highest-fidelity-wins merge.
- **Phase 3:** Relative-frame recording, anchor-relative loop playback, loop phase tracking across vessel load/unload.
- **Phase 4:** Rewind procedure, spawn-at-recording-end, warp spawn queue, PID-based deduplication, timeline immutability.
- **Phase 5:** Distance-based rendering zones, ghost soft caps with priority-based despawning.

21.2 Phase 6: Vessel Interaction Paradox Extension — COMPLETE

The ghost chain system, spawn safety, time jump, and ghost world presence are implemented. 381 new tests (2989 total). In-game testing pending for KSP runtime paths.

Phase	Scope	Status
-------	-------	--------

6a — Ghost Chain Infrastructure	Chain walker, claiming logic, intermediate spawn suppression, PID preservation	Done (105 tests)
6b — Ghost Conversion + Chain-Aware Spawn	Real-to-ghost conversion, chain-aware spawn-at-end, save/load re-evaluation	Done (79 tests)
6c — Spawn Safety	Bounding box collision, ghost extension, terrain correction, trajectory walkback	Done (93 tests)
6d — UI	Spawn warnings, ghost labels, chain status display	Done (27 tests)
6e — Relative-State Time Jump	Discrete UT skip, TIME_JUMP event	Done (27 tests)
6f — Ghost World Presence	Map view, tracking station, CommNet relay (stock API), antenna specs	Done (47 tests)

New source files: GhostingTriggerClassifier, GhostChain, GhostChainWalker, VesselGhoster, SpawnCollisionDetector, GhostExtender, TerrainCorrector, SpawnWarningUI, TimeJumpManager, GhostMapPresence, GhostCommNetRelay, AntennaSpec. Recording format version bumped to 7 (additive TerrainHeightAtEnd field).

21.3 Deferred Items

Item	Reason
Full ghost visual treatment (transparency, outlines)	Ghost labels provide minimum viable distinction. Full treatment is polish.
Resource transfer tracking	Moot in stock KSP (requires docking, already a MERGE trigger).
Chain walker caching	Trivial cost for typical tree sizes.
Manual placement UI for total trajectory overlap	Rare edge case.
Quicksave pruning	Storage management only, no architectural impact.

Consolidated from: recording-system-design.md, parsek-vessel-interaction-paradox.md, parsek-vessel-interaction-paradox-addendum-time-jump.md, and recording-system-extension-plan.md.